

Nano4 GP1 生醫 HPC 課程

第三堂：進階與真實案例

第 05 章 AI Agent 自動化 Slurm 排程

第 06 章 AI Agent 技能總匯庫 (Skills Hub)

第 07 章 nf-core/ampliseq 真實 16S 案例

補充 課程回顧與延伸資源

 線上講義：<https://gemini960114.github.io/Nano4-Docs/>

今天的時間表

時間	內容
0:00-0:10	複習、重新連線
0:10-0:25	先送出第 07 章作業 （下載資料、官方測試）
0:25-0:35	第 07 章故事：一位研究生的一天
0:35-0:50	Apptainer 容器（第 03 章 Lab 9）
0:50-1:15	第 05 章：請 AI 把流程改寫成 Slurm 管線
1:15-1:25	☕ 休息（順便看一次第 07 章進度）
1:25-1:45	第 06 章：安裝課程 Skills，用 Skill 抓錯
1:45-2:05	第 06 章：比較我的 skill 與課程 Skills
2:05-2:15	檢查官方測試結果
2:15-2:50	講師示範：解讀 ampliseq 真實資料結果
2:50-3:00	總結、說明繳交內容

今天結束時，你會做到

1. 在計算節點用 **Apptainer** 容器執行分析，不需要 `module load`
2. 請 AI Agent 把分析流程改寫成**離線與外網直連**兩種 Slurm 作業
3. 用 Skill 抓出 `sbatch --test-only` 漏掉的錯誤
4. 比較你的 **skill** 和課程 Skills，把值得學的補進自己的 skill
5. 看懂 nf-core/ampliseq 的官方測試與真實 16S 腸道菌相分析結果

上課前：重新連線 🖐️

#	項目	完成了嗎？
1	在自己電腦啟動 ssh-proxy ，完成一次密碼與 OTP	☐
2	Antigravity 用 <code>Connect to Host...</code> 連上 nano4-proxy	☐
3	更新課程教材（指令如下）	☐
4	第二堂做的 <code>~/.agents/skills/my-nano4-slurm/SKILL.md</code> 還在	☐

```
cd "$HOME/Nano4-Docs"  
git pull --ff-only
```

本課程的固定設定（請記住）

計畫代號：GOV115088（國網生技醫藥高效能運算推廣與應用計畫）

佇列：ngs62g

規格：每個作業固定 -c 8 --mem=62G（國網官方規定，不可自行縮減）

環境：CPU-only，本次不使用 GPU

第 07 章的 Nextflow 主控作業與所有子工作，也都送到 ngs62g，每個都申請 8 核心 / 62 GB。

開課先做

送出第 07 章的作業，讓它在背景跑



先送出 ①：先備檢查

```
cd "$HOME/Nano4-Docs/07-nfcore-ampliseq-case-study"  
hostname  
wallet G0V115088  
module load biology/nf-core-ampliseq/2.18.0  
nextflow -version  
singularity --version || apptainer --version  
module purge
```

執行唯讀 preflight，確認 G0V115088 與 ngs62g 的授權：

```
bash "$HOME/Nano4-Docs/06-skills-hub/nano4-slurm-operations/scripts/slurm-preflight.sh" \  
  --project G0V115088 \  
  --partition ngs62g
```



先送出 ②：下載並驗證真實 FASTQ（約 5 分鐘）

在計算節點下載約 286 MB 的 ENA FASTQ，逐檔核對 MD5，並建立 samplesheet：

```
export NFCORE_ACCOUNT=GOV115088
export NFCORE_PARTITION=ngs62g

sbatch \
  --account="${NFCORE_ACCOUNT}" \
  --partition="${NFCORE_PARTITION}" \
  scripts/01_prepare_real_data.slurm
```

提交後記下 Job ID，只做一次狀態快照：

```
squeue -j '<JOB_ID>'
```

 `.err` 日誌出現 `Will retry` 是正常的：ENA 偶爾中途斷線，腳本會自動續傳。



先送出 ③：官方 test profile （約 28 分鐘）

```
export NFCORE_ACCOUNT=GOV115088
export NFCORE_PARTITION=ngs62g

sbatch \
  --account="${NFCORE_ACCOUNT}" \
  --partition="${NFCORE_PARTITION}" \
  --export=ALL,NFCORE_ACCOUNT,NFCORE_PARTITION \
  scripts/02_run_official_test.slurm
```

```
Slurm driver job
├── Nextflow
│   ├── Slurm process: FastQC
│   ├── Slurm process: Cutadapt
│   ├── Slurm process: DADA2
│   └── Slurm process: QIIME2 / MultiQC / reports
```

✓ 送出後就不用管它，等一下再回來看結果

執行時你會看到什麼？

只執行一次 `queue --me` 查看（不要用 `watch` 反覆刷新）：

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
425914	ngs62g	ampliseq	student	R	03:12	1	25a-cpn01
425930	ngs62g	nf-NFCOR	student	R	00:41	1	25a-cpn01
425931	ngs62g	nf-NFCOR	student	PD	00:00	1	(Priority)

你看到的	意思
<code>ampliseq</code> （ <code>ampliseq_test</code> ）	主控作業： 裡面跑 Nextflow ，一直執行到整條 pipeline 結束
<code>nf-NFCOR</code> （ <code>nf-NFCORE_AMPLISEQ_...</code> ）	子作業： pipeline 的某一步，會陸續出現、結束，可達上百個
<code>PD + (Priority) / (Resources)</code>	排隊等資源，屬正常現象

第 07 章故事：一位研究生的一天

- **研究問題：**幼鼠經歷早期壓力（MSEW：母鼠分離＋提早斷奶）後，腸道菌相會不會改變？
- **方法：**糞便樣本做 **16S rRNA 擴增子定序**，每條 read 就像細菌的「條碼」
- **任務：**用作者公開的原始資料，在 Nano4 上把分析重現一次
- **課堂版本：**6 個樣本（3 個 Control、3 個 MSEW），分組完全照作者的 metadata

小林的想法	對應章節
先搞懂資料從哪來	§1 、 §3.1
資料要下載完整、沒損壞	§3.2
先確認機器這關能過	§3.3 官方 test
環境沒問題了，才跑真實資料	§3.4
讀懂結果，不要過度解讀	§5
請 AI 再做一次，逐項檢查	§4
把做對的規則留下來	§6 Skill

為什麼一定要先跑 test ?

分析失敗的原因只有兩大類：

類型	例子
環境問題	軟體、容器、Slurm 設定、權限
資料問題	檔案損壞、primer 錯誤、樣本表寫錯

- **test 成功** → 環境沒問題，之後失敗就專心檢查資料與參數
- **test 失敗** → 問題一定在環境

 test 資料是「軟體測試用」，它的分組與圖表**不能**拿來做生物學解讀。



Lab 9：用 Apptainer 容器執行 MultiQC（1/2）

先執行第 04 章 §5 的對照組（約 15 秒），產生 FastQC 結果：

```
cd "$HOME/Nano4-Docs/04-ai-assisted-bio-pipeline/scripts"  
bash download_demo_fastq.sh  
bash run_fastqc_multiqc.sh
```

再送出容器作業（不載入任何模組，MultiQC 裝在容器裡）：

```
cd "$HOME/Nano4-Docs/03-slurm-syntax-and-job-management"  
sbatch templates/singularity_job.slurm
```

作業會依序：確認主機上沒有 `multiqc` → 從 Docker Hub 拉取 `multiqc/multiqc:v1.35` → 在容器內執行 MultiQC。



Lab 9：查看結果（2/2）

```
cat apptainer_multiqc-*.out
```

```
[1] 主機上的 multiqc：找不到（正常，沒有載入模組）
[2] 拉取映像檔 docker://multiqc/multiqc:v1.35 ...
[3] 容器內的 MultiQC：multiqc, version 1.35
[4] 以容器執行 MultiQC ...
報告位置      : /work/<帳號>/apptainer_lab/multiqc_out/multiqc_report.html
```

- 報告內容和第 04 章用模組產生的相同，但這次**完全沒有 module load**
- nf-core 也是用容器運作：這正是第 07 章在背景做的事



出現 **toomanyrequests**：Docker Hub 下載次數暫時到上限，10-15 分鐘後再送一次。

第 05 章

AI Agent 自動化 Slurm 排程

第 05 章： 7 個小節

§	主題	本堂課
1	為什麼要派送到 Slurm	講解
2	請 AI Agent 重構 Slurm 腳本	實作
3	兩種架構：離線運算 vs 外網直連	講解
4	 Case A：事前下載、離線運算	實作
5	 Case B：計算節點外網直連	講師示範
6	結果檢驗與 seff	實作
7	課程進度回顧	—

§1 為什麼要派送到 Slurm ?

第 04 章 §5 的對照組在登入節點跑 `run_fastqc_multiqc.sh`，但：

- 登入節點多人共用，系統會限制個人 CPU 與記憶體
- 只有送到計算節點，才能申請：
 - **多核心 CPU**：ngs62g 每個作業 8 核心
 - **大記憶體**：ngs62g 每個作業 62 GB
- 生醫平台計畫另有數百核心、最高 6 TB 記憶體的佇列（本課程不使用）

§2 請 AI Agent 重構 Slurm 腳本

在 Antigravity 以 `$HOME/Nano4-Docs` 為工作資料夾，開啟任一個 Agent：

你是一位熟悉國網中心 Nano4（晶創 26）超級電腦 Slurm 排程器與生物資訊分析的專家。
我原本在登入節點有一個執行 FASTQ 質控分析（FastQC + MultiQC）的互動腳本 ``run_fastqc_multiqc.sh``。
現在我希望將這套流程改由 Slurm 佇列派送到 Nano4 計算節點（Compute Node）執行。

請幫我編寫兩個版本的 Slurm 批次作業腳本（符合 Nano4 規格，本課程計畫使用 `#SBATCH --account=GOV115088` 與 `--partition=ngs62g`，並依 ngs62g 官方規格加上 `#SBATCH --cpus-per-task=8` 與 `#SBATCH --mem=62G`）：

1. 案例 A：事前資料下載 / 離線運算模式（資料已在 `/work` 高速目錄就緒，計算節點純內網多核平行處理）。
2. 案例 B：外網直連 / 動態下載模式（利用 Nano4 計算節點 Direct Internet 存取能力，即時抓取遠端資料並質控）。

 在 `Nano4-Docs` 裡，你的 `my-nano4-slurm` 和課程 Skills 都會一起載入

§3 兩種架構：離線運算 vs 外網直連

【架構 A：事前下載 / 離線運算】

登入節點下載 FASTQ → /work/\$USER → 計算節點讀取 /work，多核平行分析

【架構 B：計算節點外網直連】

Slurm 派送作業 → 計算節點直接連外網下載 → 分析 → 結果寫入 /work/\$USER

比較	Case A 離線運算	Case B 外網直連
計算節點連外網	不需要	直接連線（Direct Internet）
Proxy	✗ 不需要	✗ Nano4 自帶外網，不需要
資料何時準備好	送作業前	作業開始時才下載
適合	大規模長期專案、固定參考基因組	即時抓資料、下載模型權重
穩定度	★ 最高	高（依賴外部伺服器）



§4 Case A：事前下載、離線運算

步驟 1：在登入節點準備好資料

```
cd "$HOME/Nano4-Docs/05-ai-agent-slurm-pipeline/case_a_offline"  
bash 01_download_on_login_node.sh
```

步驟 2：送出離線 Slurm 作業

```
export BIO_PROJECT_ID=GOV115088  
sbatch --account="${BIO_PROJECT_ID}" 02_submit_offline_qc.slurm
```

和 **Agent** 產生的腳本比較：`ngs62g + -c 8 --mem=62G`、`module purge` 後載入 `biology/JDK/26.0.1` `biology/FastQC/0.11.9` `biology/MultiQC`、`fastqc -t ${SLURM_CPUS_PER_TASK}` 把 8 核心用滿

§5 Case B：計算節點外網直連（講師示範）

```
export BIO_PROJECT_ID=GOV115088
cd "$HOME/Nano4-Docs/05-ai-agent-slurm-pipeline/case_b_online"
sbatch --account="${BIO_PROJECT_ID}" run_online_pipeline.slurm
```

腳本在計算節點上直接下載資料並質控：

```
# 0. 載入官方模組（計算節點沒有系統 Java）
module purge
module load biology/JDK/26.0.1 biology/FastQC/0.11.9 biology/MultiQC

# 1. 驗證計算節點對外網路連通性（無須任何 Proxy）
curl -fsSIL --connect-timeout 10 -o /dev/null "${SAMPLE_URL}"
echo "✅ 外網直連成功！"
```

 舊系統 F1 的計算節點沒有外網；Nano4 可以直接連線，不要再用舊的 Proxy 指令。

§6 結果檢驗與 seff

```
queue -u $(whoami)
```

作業從清單中消失後，查看日誌與報告：

```
# 案例 A 日誌與報告：  
cat "$HOME/Nano4-Docs/05-ai-agent-slurm-pipeline/case_a_offline"/qc_offline-*.out  
ls /work/${USER}/nano4-case-a-qc/multiqc_out_offline/  
  
# 案例 B 日誌與報告：  
cat "$HOME/Nano4-Docs/05-ai-agent-slurm-pipeline/case_b_online"/qc_online-*.out  
ls /work/${USER}/nano4-case-b-qc/multiqc_out_online/
```

```
seff <JOB_ID>
```

休息 10 分鐘

執行一次 `queue --me`，看看第 07 章的作業跑到哪裡了。

第 06 章

AI Agent 技能總匯庫 (Skills Hub)

第 06 章：今天的主軸

把你的 skill 和課程 Skills 比較。

第二堂你在第 03 章 Lab 11 做了 `my-nano4-slurm`；本章的 Skills 是講師整理的版本。

課程 Skill	核心功能
<code>nano4-slurm-operations</code>	唯讀 preflight：檢查 <code>wallet</code> 、Slurm 授權與佇列政策
<code>slurm-job-advisor</code>	資源規劃、 <code>ngs62g</code> 規格檢查、 <code>validate_slurm.sh</code> 靜態檢查
<code>ai-agent-slurm-pipeline</code>	把互動式腳本改寫成 Slurm 管線、串接相依作業
<code>nfcore-ampliseq-nano4</code>	nf-core/ampliseq 的資料溯源、test、正式執行與結果檢查

安裝課程 Skills

```
cd "$HOME/Nano4-Docs/06-skills-hub"  
bash sync_skills.sh
```

複製到	給誰讀
~/.agents/skills/	Codex
~/.claude/skills/	Claude Code
~/.gemini/config/skills/	Antigravity 內建 Agent

 以 `$HOME/Nano4-Docs` 為工作資料夾時，Antigravity 與 Codex 也會直接載入 `.agents/skills/`；安裝後在其他資料夾也能用。

內建工具速查

```
# 1. 執行 Nano4 唯讀環境與計畫預檢 (nano4-slurm-operations)
bash "$HOME/Nano4-Docs/06-skills-hub/nano4-slurm-operations/scripts/slurm-preflight.sh" --project
GOV115088 --partition ngs62g

# 2. 查詢 wallet 額度與可用佇列即時資源 (slurm-job-advisor)
bash "$HOME/Nano4-Docs/06-skills-hub/slurm-job-advisor/scripts/check_slurm_env.sh"

# 3. 驗證 Slurm 腳本合規性與免扣點模擬預檢 (slurm-job-advisor)
bash "$HOME/Nano4-Docs/06-skills-hub/slurm-job-advisor/scripts/validate_slurm.sh" \
"$HOME/Nano4-Docs/03-slurm-syntax-and-job-management/templates/standard_cpu_job.slurm"
```



練習：讓 Skill 抓出不合規的腳本（1/2）

準備一份正確、兩份故意寫錯的腳本：

```
mkdir -p /work/$USER/skill_lab && cd /work/$USER/skill_lab
cp "$HOME/Nano4-Docs/03-slurm-syntax-and-job-management/templates/standard_cpu_job.slurm"
good.slurm
sed 's/^#SBATCH --mem=62G .*/#SBATCH --mem=16G/' good.slurm > bad_mem.slurm
sed 's/^#SBATCH --partition=ngs62g .*/#SBATCH --partition=ngs32g/' good.slurm > bad_partition.slurm
```

先用 `sbatch --test-only` 檢查記憶體寫錯的那份：

```
sbatch --test-only bad_mem.slurm
```

⚠ 會顯示 `Job ... to start at ...`：排程器**沒有**發現記憶體設定錯誤



練習：用 Skill 的驗證工具檢查（2/2）

```
V="$HOME/Nano4-Docs/06-skills-hub/slurm-job-advisor/scripts/validate_slurm.sh"
bash "$V" good.slurm;          echo "exit=$?"
bash "$V" bad_mem.slurm;       echo "exit=$?"
bash "$V" bad_partition.slurm; echo "exit=$?"
```

腳本	驗證結果	回傳值
good.slurm	🎉 [驗證通過]	0
bad_mem.slurm	❌ [規格錯誤] 'ngs62g' 官方規格為 --mem=62G ，目前設定為 16G !	1
bad_partition.slurm	❌ [授權錯誤] 計畫 G0V115088 只能使用 ngs62g	1

接著請 Agent 修正：

請使用 `slurm-job-advisor skill` 檢查 `bad_mem.slurm` 與 `bad_partition.slurm`，說明錯在哪裡並修正，修正後執行 `validate_slurm.sh` 確認通過，但不要提交作業。



練習：比較我的 skill 與課程 Skills （1/2）

在 Antigravity 開啟 `$HOME/Nano4-Docs`，選一個 Agent 開新對話：

請比較我的 skill `~/.agents/skills/my-nano4-slurm/SKILL.md` 和課程提供的 `06-skills-hub/slurm-job-advisor/SKILL.md`、`06-skills-hub/ai-agent-slurm-pipeline/SKILL.md`。

用表格列出：兩邊都有的規則、只有課程 Skills 有的檢查、只有我的 skill 有的經驗。先不要修改任何檔案。

和同學討論：

- 課程 Skills 有哪些檢查是你在 Lab 10 沒遇到、但真實研究會遇到的？
- 你的 skill 記下了哪些課程 Skills 沒寫的經驗？



練習：把值得學的補進自己的 skill （2/2）

請把剛才表格中「只有課程 Skills 有的檢查」裡，對我最有用的兩項補進 my-nano4-slurm，保留我原本的經驗，改完後讓我看修改的地方。

重新安裝，讓三個 Agent 都讀到新版本：

```
bash "$HOME/Nano4-Docs/03-slurm-syntax-and-job-management/scripts/install_my_skill.sh"
```



你的 skill 現在同時有自己的經驗，也有課程 Skills 的檢查

第 07 章

nf-core/ampliseq 真實 16S 案例

第 07 章：課堂安排

§	主題	本堂課
0	本章故事	已講
1	兩套資料：官方 test 與公開研究	講解
2	工作目錄與先備檢查	開課已做
3.2 / 3.3	下載資料、官方 test	開課已送出
3.4	真實資料（約 1 小時 40 分鐘）	講師示範
4	讓 AI Agent 重做	講師示範，課後完成
5	查看與解讀結果	跟著講師做
6	封裝並使用 Skill	講師示範
7	繳交內容	課後

檢查官方測試結果

```
cd "$HOME/Nano4-Docs/07-nfcore-ampliseq-case-study"  
tail -5 ampliseq_test-*.out
```

✓ 最後出現 **Pipeline completed successfully** 才代表成功（課前實測約 28 分鐘、125 個子工作、0 個失敗）

- 還在跑：先講 S1 資料來源，稍後再看
- 失敗：問題在**環境**，不是資料；檢查 **.err** 日誌

§1 本章使用的兩套資料

	官方 test profile	公開研究 PRJNA605008
用途	驗證 Nextflow、Slurm、Singularity 能不能運作	學習資料溯源與結果解讀
資料	nf-core/ampliseq 2.18.0 內建小型資料	作者公開在 ENA/SRA 的 raw FASTQ
能不能做生物學結論	✗ 不能	✓ 可以，但要注意樣本數

研究使用的 primer：

515F GTGYCAGCMGCCGCGGTAA
806R GGACTACNVGGGTWTCTAAT

§1 課堂選用的 6 個樣本

Pipeline ID	作者 SampleID	Treatment	Litter	Run
C_L1_03	C-L1-03	Control	CONT-L1	SRR11028771
C_L2_03	C-L2-03	Control	CONT-L2	SRR11028740
C_L3_04	C-L3-04	Control	CONT-L3	SRR11028776
M_L1_03	M-L1-03	MSEW	MSEW-L1	SRR11028732
M_L2_03	M-L2-03	MSEW	MSEW-L2	SRR11028720
M_L3_03	M-L3-03	MSEW	MSEW-L3	SRR11028758

- 都是作者 metadata 中的 `exp1`、`PD10`、`MALE`，分別來自三個 litter
- 分組完全照作者 metadata，課堂**沒有**自己發明分組
- nf-core/ampliseq 2.18.0 的 sample ID 不能有連字號，所以 Pipeline ID 把 `-` 改成 `_`

nf-core/ampliseq 幫我們做了什麼？

步驟	工具	白話說明
1. 品質檢查	FastQC / MultiQC	看定序品質好不好
2. 移除 primer	Cutadapt	切掉每條 read 開頭的引子（ 515F / 806R ）
3. 去雜訊、找 ASV	DADA2	過濾、校正錯誤、合併雙端 reads、去除嵌合體，得到 ASV
4. 物種分類	DADA2 + SILVA 138.2	判斷每個 ASV 是哪個門、綱、目、科、屬
5. 多樣性分析	QIIME 2	alpha：樣本內多豐富； beta：樣本間多不同（ PCoA ）
6. 總結報告	MultiQC / summary report	把各步驟統計彙整成網頁報告

 每一步都會變成一個（或數個） Slurm 子作業，送到 ngs62g。

§3.4 真實資料（講師示範）

只有在下載與官方 test 都成功後才提交：

```
cd "$HOME/Nano4-Docs/07-nfcore-ampliseq-case-study"
export NFCORE_ACCOUNT=GOV115088
export NFCORE_PARTITION=ngs62g

sbatch \
  --account="${NFCORE_ACCOUNT}" \
  --partition="${NFCORE_PARTITION}" \
  --export=ALL,NFCORE_ACCOUNT,NFCORE_PARTITION \
  scripts/03_run_real_data.slurm
```

- 固定：nf-core/ampliseq 2.18.0、研究原始 primer、作者原始 metadata、silva=138.2
- 課前實測約 **1 小時 40 分鐘**、79 個子工作；其中 QIIME 2 稀釋曲線一步就佔約 1 小時 30 分鐘，這段時間 log 不更新，不是卡住

§4 讓 AI Agent 重做（講師示範，課後完成）

人工版完成後，才用 `prompts/ai_reproduce.md` 請 AI 重做：

- AI 必須先讀研究資料、提出計畫、完成 preflight
- 你確認計畫、佇列、資源與實際命令後，AI 才能提交
- AI 版放在獨立位置 `/work/${USER}/nfcore-ampliseq-course-ai/`，不要覆寫人工版

```
# samplesheet: 人工版 vs AI 版
diff -u /work/${USER}/nfcore-ampliseq-course/metadata/samplesheet.tsv \
        /work/${USER}/nfcore-ampliseq-course-ai/metadata/samplesheet.tsv
```

比較重點： accessions 與 MD5、sample ID 是否可逆、metadata 有沒有被擅自改組、primer 與 taxonomy 是否固定、AI 是否先跑 test 並取得確認才提交

§5.1 先看講師的示範結果

 下載後在自己電腦用瀏覽器打開，不需要連上 Nano4：

<https://github.com/gemini960114/Nano4-Docs/releases/tag/ampliseq-demo-2026-09-23>

檔案	內容	建議
<code>ampliseq_summary_report.html</code>	各步驟保留的 reads、分類結果、多樣性	先看這份
<code>ampliseq_multiqc_report.html</code>	原始 reads 與 Cutadapt 品質報告	第二份
<code>overall_summary.tsv</code> 等表格	主要結果表格	可用 Excel 開啟
<code>ampliseq_results-real.tar.gz</code>	完整結果	想對照資料夾結構時再下載



§5.1 看報告時回答這 4 個問題

1. 6 個樣本分別有多少 reads 通過 Cutadapt 與 DADA2？最後得到幾個 ASV？
2. 門（phylum）與屬（genus）層級最多的是哪些？
3. Control 與 MSEW 的 Shannon 多樣性看起來有差嗎？
4. Beta diversity 的 PERMANOVA p 值是多少？可以說兩組微生物相不同嗎？

⚠️ PCoA 圖是視覺化，**不等於統計顯著**；只有 6 個樣本，不能宣稱重現了整篇論文。

§5.2 檢查自己跑出來的結果：閱讀順序

```
bash scripts/04_inspect_results.sh
```

1. `pipeline_info/`：版本、參數、process 狀態與執行時間
2. `multiqc/multiqc_report.html`：原始 reads 品質（未修剪）
3. `cutadapt/`：primer 找到率，避免 primer 填錯卻繼續分析
4. `dada2/`：filtering、denoising、merging、chimera removal 與 ASV 數量
5. `qiime2/abundance_tables/`：feature table 與 taxa abundance
6. `qiime2/diversity/`：alpha / beta diversity 與 PCoA
7. `seff <JOB_ID>`：檢查 driver job；再用 `sacct` 看子工作資源

§6 封裝並使用 Skill

課程已提供 `06-skills-hub/nfcore-ampliseq-nano4/`；安裝後，AI 在 `nf-core/ampliseq` 的任務中會自動載入：

```
cd "$HOME/Nano4-Docs/06-skills-hub"  
bash sync_skills.sh
```

Skill 封裝的是**可重用的規則**，不是把這 6 個樣本寫死：

資料溯源與 MD5

- primer 與實際 reads 核對
- sample/metadata 驗證
- Nano4 live preflight
- 官方 test profile
- 使用者確認後正式提交
- 結果與資源檢查

§7 繳交內容（課後）

1. manual-run-summary.md
人工操作、Job IDs、版本、參數與主要結果
2. ai-run-summary.md
AI 的計畫、人工核准點、Job IDs 與主要結果
3. comparison-report.md
人工版和 AI 版的差異、錯誤與風險
4. skill-review.md
哪些規則適合封裝、哪些生物學判斷仍需人工負責
5. my-nano4-slurm/SKILL.md
第 03 章做的 skill，含第 06 章比較後補強的內容

👉 目標不只是「會跑 nf-core」，而是能理解、監督、驗證並重用一套真實的生物資訊 workflow

補充

課程回顧與延伸資源

三堂課回顧

堂	主題	你帶走的東西
第一堂	連線與 AI Agent	Antigravity + ssh-proxy 只認證一次；三個 AI Agent ； AGENTS.md
第二堂	Slurm 與我的 skill	標準、陣列、相依作業； seff ； 自己的 skill my-nano4-slurm
第三堂	進階與真實案例	容器；離線與外網直連管線； Skill 抓錯； nf-core 16S 真實分析

貫穿三堂的一件事： AI Agent 替你寫指令、送作業、解讀結果；**你負責確認**它的計畫與結果是否正確。

附錄：給 AI 的 Nano4 守則（AGENTS.md）

1. 禁止 `sudo`：軟體改用 `module`、`uv` 或 `Apptainer` 容器
 2. 大資料放 `/work/$USER`：不塞爆 `$HOME`，不寫 `/tmp`
 3. 登入節點不跑重度運算：一律封裝成 `Slurm` 作業
 4. `Slurm` 標準：`module purge` 放第一行、日誌用 `%x-%j.out`、不反覆查詢 `squeue`
 5. 本課程設定：`GOV115088` 只能用 `ngs62g`，固定 `-c 8 --mem=62G`
 6. 替使用者操作：先用白話說明計畫、等你同意再送出，跑完用白話解讀結果
-  回到自己的研究時，把 `AGENTS.md` 複製到專案資料夾，改成自己的計畫與佇列

資源與排隊：全班同時送作業時

- ngs62g 共 13 個節點、1,664 個核心，大約可同時執行 **200 個** 8 核心作業
- 每位使用者最多 **120 個** 作業
- 第 07 章官方測試每人約 1 + 5 個作業，20 位學員約 120 個，沒問題
- 真實資料（§3.4）不要全班同時跑，改看講師的示範結果

⚠ 作業一直 PD 且原因是 (Priority) 或 (Resources)：在排隊，屬正常現象，不要重複送出

回到自己的研究：從這裡開始

1. 用 `wallet` 確認自己的計畫，再用 `scontrol show partition` 查能用哪些佇列與規格
 2. 把 `AGENTS.md` 改成自己的計畫、佇列與固定規格，放進專案資料夾
 3. 先用小樣本在 Slurm 上跑通，再放大到全部資料
 4. 每次請 AI 做完，檢查 `seff`、日誌與輸出檔數量
 5. 把糾正 AI 的經驗補進自己的 skill
-  生醫平台計畫（如 `MST109178`）可使用更多 `ngs*` 佇列與大記憶體節點，規格同樣是固定搭配。

官方參考文件

- GP1 生醫核心設施使用說明: https://man.twcc.ai/x0YzPATVS_aDlbuqMrwhyg
- Nano4 使用者操作手冊: <https://man.twcc.ai/@nano4-manual/documentation>
- Nano4 Slurm 佇列與資源規格: https://man.twcc.ai/@nano4-manual/SJM_FuxDWe
- Nano4 Slurm Job 提交與管理範例:
https://man.twcc.ai/@nano4-manual/BkRXxZ_JMg
- Nano4 儲存資源與目錄位置: <https://man.twcc.ai/@nano4-manual/ry1hWDlPbl>
- 課程教材原始碼: <https://github.com/gemini960114/Nano4-Docs>

課程結束

📖 線上講義: <https://gemini960114.github.io/Nano4-Docs/>

📦 第 07 章示範結果:

<https://github.com/gemini960114/Nano4-Docs/releases/tag/ampliseq-demo-2026-09-23>

🔒 離開前在 proxy 視窗按 **Ctrl+C** (尤其是共用電腦)

謝謝大家! 🙌